Lightweight confidential and traceable multi-stakeholder data processing

Steylor Barros[1][?????], Eduardo Falcão[2][0000−zzz−zzz−zzzz], and Andrey Brito[1][0000−0002−7350−8599]

[1] Universidade Federal de Campina Grande, Campina Grande/PB, Brazil
[2] Universidade Federal do Rio Grande do Norte, Natal/RN, Brazil
steylor.barros@ccc.ufcg.edu.br
eduardo@dca.ufrn.br
andrey@computacao.ufcg.edu.br
https://www.lsd.ufcg.edu.br/

Abstract. We leverage technologies such as confidential computing and distributed ledgers to propose an architecture for multi-stakeholder, confidential data processing. Our approach aims to protect non-interactive data-processing applications in scenarios where multiple parties (e.g., data controllers, application developers, and infrastructure managers) distrust one another. We generalize previous results, describe the architecture and requirements of the involved components, and present an implementation focused on lightweight execution in the AWS cloud. Our implementation focuses on Python applications, showing overheads as low as $X\%$.

Keywords: Multi-stakeholder data processing · Privilege separation · Lightweight virtualization (microVMs) · Confidential computing · Distributed ledger · Traceability · Supply-chain security

1 Introduction

Confidential data marketplaces must simultaneously support asset exchange, access control, and third-party computation over sensitive inputs—a combination that creates a difficult systems problem. A marketplace that accepts externally supplied applications cannot treat code execution as an isolated implementation detail, because the same platform also handles persistent metadata, payment and access logic, storage backends, and the datasets that motivate the marketplace in the first place. The problem is therefore not merely how to "run code safely," but how to prevent build-time and run-time compromise from expanding into a full platform compromise.

The most direct implementation path would be a centralized confidential virtual machine [1] hosting the marketplace-facing logic, local artifact storage, and the execution orchestrator in one trust domain. That design minimizes deployment complexity and avoids inter-service coordination, but it creates the wrong security shape for the problem: the same node that validates access and manages

persistent sensitive state also becomes the place where untrusted or semi-trusted workloads are prepared and executed.

This paper argues that such a concentration of privilege is fundamentally ill-suited for high-assurance marketplaces, and proposes an architecture designed under the assumption that component compromise is an eventual certainty rather than a remote possibility. By decomposing the marketplace into distinct trust domains—control, build, and execution [6]—a failure or malicious exploit in one phase cannot propagate to the whole platform. The focus is on minimizing the "blast radius" of errors and attacks: even if the build or execution planes are corrupted, the core sensitive assets and persistent state remain protected.

The paper proceeds as follows. Section 2 introduces the three building blocks the architecture rests on—lightweight isolation, distributed ledgers, and distributed storage—each by the property it contributes, and Section 3 defines the problem and derives the requirements R1–R6 a secure solution must satisfy. Section 4 presents the three-plane architecture, its threat model, and how it generalizes the SGX-based DNAT; Section 5 describes the prototype that realizes the planes. Section 6 evaluates the design for security (a STRIDE analysis mapped to reproducible tests) and cost (a live A/B comparison against a centralized baseline). Section 7 positions the work against related trusted-execution and ledger-based systems, and Section 8 concludes.

2 Background

The architecture combines three building blocks, each introduced here by the property it contributes rather than by its internal mechanics.

2.1 Lightweight isolation with microVMs

The platform must execute application code and resolve third-party dependencies—both only partially trusted—without letting a compromise spread across trust domains. Operating-system containers share the host kernel and offer a comparatively weak boundary for this purpose, whereas full virtual machines impose a startup and memory cost hard to justify per task. MicroVMs such as Firecracker [2] occupy the middle ground: each workload runs behind a virtualization boundary with its own kernel, but with a minimal device model that keeps startup time and memory footprint low. Two levers thus become practical at the granularity of a single execution. *Ephemerality*: a fresh microVM per task leaves no residual state for a later run to inherit. *Network confinement*: the environment that consumes the sensitive dataset can be launched with no network interface at all, removing the primary exfiltration channel even under a complete runtime compromise.

2.2 Distributed ledgers and traceability

A distributed ledger [5] provides a tamper-evident, append-only record shared among parties that do not trust one another. This is what yields traceability:

every consequential action—registering an asset, purchasing access, granting a permission—is recorded as a transaction that no single party can later repudiate or rewrite. Smart contracts [4] turn the access policy itself into an executable, publicly verifiable program: the ledger holds the access rights and the content hashes of the registered assets, so any stakeholder can verify what was authorized and which artifact was used. The cost is that every write requires consensus, adding a fee and latency—which is precisely why the large assets must live off the ledger.

2.3 Distributed storage

Those assets—encrypted application and dataset artifacts—require an off-chain store that is *content-addressed*, so that the identifier of an artifact is a deterministic hash of its bytes and can be bound to the integrity record on the ledger, and *distributed*, so that the data stays available without a single trusted custodian. Any storage layer meeting these two objectives is compatible with the architecture. We use the InterPlanetary File System (IPFS) [3] in a deliberately simplified form—a node co-located with the control plane rather than a public swarm—as a stand-in for any such backend.

3 Problem definition and solution requirements

The core challenge involves a transaction between two mutually distrusting parties: a *data buyer*, who owns a confidential application, and a *dataset provider*, who holds sensitive data. The objective is a computation in which the buyer's application processes the provider's dataset without exposing either asset to the other party or to the cloud infrastructure.

In the monolithic CVM sketched above—one trust domain receiving uploads, managing the storage layer, driving the smart contract, resolving dependencies, and running the application—the critical vulnerability emerges during dependency resolution: installing external packages necessarily executes arbitrary third-party code with the same privileges as the marketplace layer, the IPFS node, and the blockchain client. A maliciously crafted package—or a compromised dependency in the supply chain—could exploit this phase to exfiltrate stored datasets, tamper with marketplace metadata, or establish a persistent backdoor affecting future executions. A build-time compromise thus propagates immediately into a full platform compromise, making dependency resolution a disproportionately dangerous entry point.

This reframes the core problem. Securing the application at runtime is necessary but not sufficient: a compromise during build and dependency resolution must also be prevented from reaching the broader platform. The problem must therefore be treated as a *privilege separation problem*: the operational phases—control, build, and execution—carry fundamentally different risk profiles and must occupy separate trust domains with different network-access policies, persistence rights, and failure blast radii. A secure solution must satisfy the following requirements.

- **R1 (Asset confidentiality).** Neither the buyer's application nor the provider's dataset is revealed to the other party or to the infrastructure operator.
- **R2 (Privilege separation).** No single component holds the privileges of all phases; a compromise of one plane must not extend to the assets of the others.
- **R3 (Build isolation).** Dependency resolution—which runs partially trusted third-party code with network access—must be isolated from persistent sensitive storage and from the execution runtime.
- **R4 (Execution containment).** The plane that consumes the dataset must have no outbound network and keep no state across runs, so that a runtime compromise can neither exfiltrate nor persist.
- **R5 (Traceability and access control).** Asset registration, access grants, and artifact integrity must be recorded on a tamper-evident ledger, and execution must be gated by a verifiable access check.
- **R6 (Application generality).** The platform must run unmodified application code with arbitrary declared dependencies, without recompilation or enclave-specific adaptation.

Section 6 returns to these requirements: R1–R5 are exercised by the STRIDE analysis (Table 1) and R6 by the cross-architecture comparison (Table 2).

4 Proposal

This section presents the architecture as a response to requirements R1–R6.

4.1 Threat model and trust assumptions

The system mediates between mutually distrusting parties: a dataset provider, an application buyer, and the marketplace operator. We assume an adversary that can submit a malicious application, declare a malicious or compromised dependency, or take control of an individual plane at runtime; the design goal is to bound what such an adversary reaches. Three capabilities are therefore in scope: malicious code at execution time, malicious code at build time (supply chain), and a fully compromised plane.

We treat the following as trusted. The distributed ledger executes its smart contracts correctly and is tamper-evident, so that recorded access rights and content hashes are authoritative. The virtual-machine monitor and the host enforce the microVM boundary; in the lightweight model targeted by the prototype this extends to trusting the cloud operator, so the boundary being defended is the one between the planes, not the one against the operator. A production deployment removes the operator from the trusted base by running each plane inside a confidential VM (e.g., AMD SEV-SNP [1]), which adds memory encryption and remote attestation without changing the architecture. Out of scope are micro-architectural side channels, load-induced denial of service, and—in the single-host prototype—the shared control network among planes, reported as a residual in Section 6.

4.2 Architecture

The architecture composes the building blocks of Section 2 into three planes, each a separate trust domain matching one of the operational phases of Section 3—control, build, and execution—so that the different risk profiles of those phases are met by different containment. The principle throughout is that a plane holds only what it needs and reaches only what it must, and the division of state follows from it. The control plane (CVM1) is the only stateful, externally reachable component—it holds the encryption and wallet keys, the storage repository, and the marketplace state—and it never resolves dependencies nor runs user code. The build plane (CVM2) is the only plane with general network egress, because resolving dependencies requires reaching package indexes, and its only durable state is a dependency cache that is read-only at use. The execution plane (CVM3), which consumes the dataset, is the most constrained: no network interface and no persistent state, a fresh microVM per run.

Figure 1 traces a single end-to-end request over the numbered interactions I1–I8, which also serve as the unit of analysis for the STRIDE evaluation. Registration and purchase go through the control plane, which encrypts each artifact, publishes the ciphertext to the store (I3) and records its content hash and the resulting grant on the ledger (I1–I2). An execution request is first checked against that grant on chain (I2); only then is a build bundle assembled and forwarded to the build plane (I4), which resolves dependencies through filtered egress (I5) and returns a read-only artifact (I6). The control plane then provisions input and output disks, attaches the artifact read-only, and launches the networkless execution microVM (I7), reading the result back from the output disk (I8) before tearing the working state down. Because each boundary crossing carries only what the next phase needs—a bundle to the builder, an artifact and disks to the executor—a compromise on either side of a boundary cannot widen into the privileges of the other.

4.3 From DNAT to a generalized architecture

The original DNAT [6] addresses the same marketplace problem but binds it to a single substrate: applications run inside SGX enclaves, a process-level TEE that constrains both the threat model and the kind of code that can run. Our contribution is to restate the problem as one of privilege separation across operational phases—independent of the isolation technology—replacing the enclave-centric decomposition with explicit control, build, and execution planes whose blast radius is bounded per plane. Two consequences follow. By shipping dependencies as ordinary packages inside a read-only artifact rather than recompiling them into an enclave, the platform runs unmodified applications with arbitrary declared dependencies (R6), which the enclave model does not. And because the architecture is defined over roles rather than over a specific trusted-execution technology, it admits different instantiations: the prototype trusts a confidential-computing root (the cloud), which is what buys the dependency freedom, whereas DNAT minimizes trust in the platform—including the operator—at the cost of that

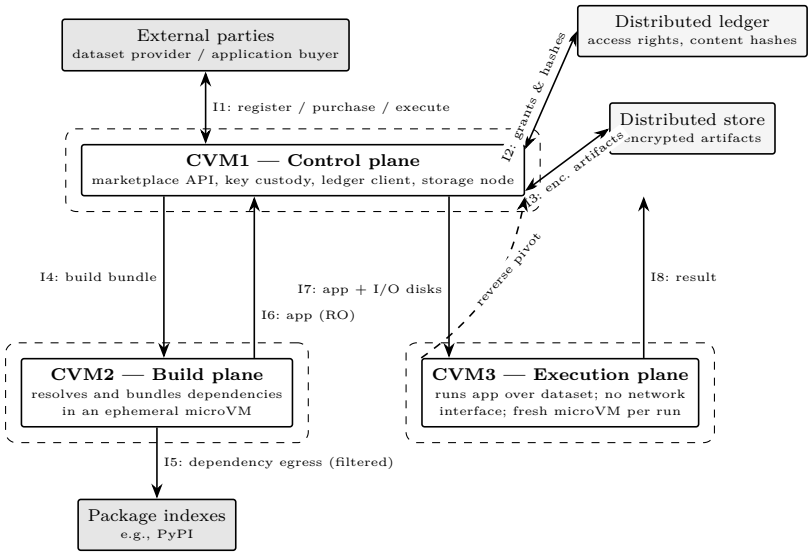

Fig. 1. The three planes and their numbered interactions (I1–I8). Dashed boxes mark trust boundaries; the STRIDE analysis of Section 6 is performed over these elements and interactions. The dashed arrow marks the reverse pivot (a compromised executor reaching the control plane), the residual quantified in Table 3.

freedom. DNAT is thus one point in a design space that this architecture makes explicit.

5 Implementation

This section describes how the three planes are realized, so that the security properties evaluated later can be traced to concrete mechanisms rather than to design intent alone. The system is deployed as three containers on a shared runtime—`dnat-client` (CVM1), `dnat-builder` (CVM2), and `dnat-executor` (CVM3)—each standing in for the hardware-backed confidential VM [1] of a production deployment, and is contrasted with two baselines: a single-container deployment that colocates the same assets (A) and a no-isolation variant that removes the microVM entirely (M0). The consequences of that substrate substitution are examined in Section 6.

5.1 Control Plane (CVM1)

The control plane hosts the marketplace API, a local IPFS node, and the blockchain client, exposing registration, access checks, and execution orchestration through a small HTTP API. On registration, CVM1 does not install dependencies itself: it assembles a build bundle (the source plus a manifest of declared dependencies) and forwards it to the builder. When the builder returns an `application.ext4`

artifact, CVM1 encrypts it with AES-256-GCM over a gzip-compressed image, in a versioned envelope carrying the IV, authentication tag, and metadata. Only the encrypted blob and a manifest are published to IPFS; the plaintext artifact never leaves the control plane. Access control is enforced on chain: an execution request first calls `hasAccessByIds`, and the workflow aborts with *access denied* if the caller has not purchased access to both assets. A purchased right is recorded under the key `keccak256(encryptedDatasetHash, encryptedApplicationHash, buyer)`—bound to the encrypted IPFS CIDs of the two assets and the buyer's address rather than to mutable identifiers, so the grant is stable under off-chain re-indexing—and is verified before any bundle is assembled.

5.2 Build Plane (CVM2)

The builder exposes a single `/build` endpoint that accepts a bundle and launches an ephemeral Firecracker microVM to resolve dependencies. Unlike the executor, the build microVM is given a network interface, because resolving and compiling Python wheels may require egress to package indexes. Egress is constrained on the host side: a TAP device is created and iptables rules `REJECT` traffic from the guest toward private ranges (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, `127.0.0.0/8`, `169.254.0.0/16`, and the host TAP address), while the remaining outbound traffic is masqueraded. Inside the guest, an artifact builder produces a read-only `ext4` image holding the application source, its `site-packages`, a lock file of resolved dependencies, and a marker file used later for disk discovery. Dependencies are first wheel-built and then installed with `-no-index` from the local wheelhouse; reusable wheels are persisted in a size-pruned cache that is the builder's only durable state—and, as Section 6 notes, its only cross-request channel. After the build, the host returns only the artifact and its metadata; the per-request build state is discarded.

5.3 Execution Plane (CVM3)

The executor exposes a single `/execute` endpoint. For each request it copies a fresh root overlay, creates new input and output `ext4` disks, optionally attaches the application artifact read-only, and launches a Firecracker microVM with one vCPU, 512 MiB of memory and—crucially—no network interface. The guest does not receive disk roles out of band; it discovers them by scanning candidate block devices for marker files, then extracts the workspace and runs `workspace/run.sh`, whose output is persisted to the output disk. Host and guest synchronize over the serial console: the guest writes an `EXECUTION_COMPLETE` marker, after which the host issues a graceful shutdown, escalating to termination if needed, then mounts the output disk and derives the HTTP response solely from the persisted result. A `cleanup` trap unmounts every loop device and removes the per-run working directory on exit, which is what makes execution state ephemeral.

5.4 Application Contract, Asset Integrity, and Baselines

An application is registered as unmodified source together with a manifest of its third-party dependencies, which the builder resolves and bundles into the read-only `application.ext4` artifact. Because dependencies are shipped as ordinary `site-packages` inside that artifact—rather than linked into a trusted-execution enclave at build time—an application that follows this contract runs with arbitrary declared dependencies, unchanged across all three environments and without recompilation or enclave-specific adaptation (R6). This is the reuse property that Section 6 contrasts with the enclave-based original DNAT.

Assets are bound to the chain by content hash: registration records the SHA-256 of the artifact, which lets a consumer detect tampering between IPFS and the contract. The first baseline (environment A) runs the identical control, build, and execution logic, including Firecracker isolation for both microVMs, but inside a single container whose filesystem holds the IPFS repository, the wheel cache, and the executions store, and whose environment holds the asset-encryption key and the wallet private key. Its only structural difference from the compartmentalized deployment is the colocation of these assets—exactly the variable the security evaluation isolates.

The second baseline removes isolation altogether. With `DNAT_NO_MICROVM` set, the executor runs the same `workspace/run.sh` contract directly in the container's namespace—no Firecracker, no per-run disks, no network removal—while emitting the same result, so the request path is otherwise unchanged. In this variant (M0) the application inherits the container's network, environment (including the control-plane secrets), and persistent filesystem, which is what lets the evaluation separate the microVM's operational cost from the containment it provides.

6 Evaluation

The architectural argument of this paper is comparative: the compartmentalized design should contain compromise better than less-separated alternatives while paying an acceptable operational cost. We evaluate it along two axes. For *security*, we organize the analysis with the STRIDE model [7], applied per interaction over the elements of Figure 1, showing how the design neutralizes—or explicitly fails to neutralize—each non-trivial threat, and mapping every claim to a concrete test. For *cost*, we measure execution and build latency. The comparison spans up to four points on an isolation spectrum (Table 2): a no-isolation monolith (M0), a centralized deployment with ephemeral microVMs but colocated assets (Environment A), the compartmentalized design that also separates the planes (Environment B), and, as a cited reference, the SGX-based original DNAT [6]. Because A and B share the same Firecracker microVMs, any difference between them isolates the effect of *compartmentalization*; M0 isolates the cost and the security delta of the *microVM* primitive itself; and the SGX reference frames the reuse limitation that motivates our change of technology.

6.1 Methodology

We exercise the running prototype with a small suite of purpose-built applications, each designed to attack one STRIDE category, over a synthetic, PII-free dataset of sixty rows. Tests targeting the execution plane submit an application bundle directly to the executor, reproducing the contract the control plane generates, so that the variable under study—the execution microVM—is isolated from unrelated services; tests targeting lateral movement inspect the executor environment, and the access-control test drives the marketplace API. Each adverse application emits machine-readable evidence markers that a harness collects, and each test is mapped to the interaction and STRIDE category it supports (Table 1), so that every observation connects to a specific boundary crossing rather than merely listing successful checks.

The suite comprises eight probes, referred to by code throughout this section so that every "Tn" entry in Tables 1 and 3 traces back to a concrete, reproducible adversarial application. **T1** and **T1b** drive the execution plane (I7): the first attempts outbound network exfiltration of the dataset, the second scans the environment and the disks for control-plane secrets and sibling data. **T2** declares a malicious dependency whose import-time payload runs inside the execution microVM (I4/I5), with two variants—**T2a**, which retries exfiltration from that payload at run time (I7), and **T2b**, whose build-time `setup.py` tries to poison the read-only wheel cache and artifact (I6/I7). **T3** repeats executions to probe the executor for residual sockets, disks, and mounts. **T4** has a compromised executor pivot toward the control plane's secrets and stores. **T5**, the only end-to-end test, drives the marketplace API (I1–I3): it requests an execution without `purchaseAccess` and audits that the actions were recorded on chain and that the on-chain hash matches the artifact. All three environments were measured live on the same host, and the gas cost on Hardhat; only the SGX column of Table 2 is cited rather than measured.

6.2 Security Evaluation with STRIDE

STRIDE analyzes components and their interactions, not a system taken as a whole—a claim such as "the design resists tampering" is meaningful only relative to a stated element—so we fix the level of abstraction first. The analysis is performed over the elements of Figure 1 and over the numbered interactions I1–I8 that cross their trust boundaries. Following STRIDE-per-element practice [7], each of the six threat categories was examined against every element and boundary-crossing interaction; flows internal to a single trust domain are not analyzed, and microVM or VMM escape at I7 is excluded by the trusted-VMM assumption of Section 4. Every cell of that sweep falls into one of four groups: actively countered by a design mechanism, trivially handled by standard TLS or by a trust assumption, an acknowledged residual, or not applicable. Table 1 concentrates on the first group—the non-trivial threats the architecture must actively counter—naming for each the interaction where it arises, the countering mechanism, and the test that provides evidence. The other three groups

are enumerated at the end of this subsection, so that the coverage can be audited rather than taken on trust.

Table 1. Non-trivial STRIDE threats, per interaction of Figure 1 (S/T/R/I/D/E = spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege). Trivial and assumption-covered cells are discussed in the text.

Interaction	Cat.	Threat & countering mechanism	Outcome (measured)	Test
I1	S	Caller without access rights; on-chain gate (`hasAccessByIds`) before any bundle is prepared	Run rejected, *access denied*, before code or data move	T5
I1, I2	R	Denial of a registration or purchase; recorded as ledger transactions	Both actions recorded on chain, auditable	T5
I3	T	Artifact modified between storage and use; SHA-256 content hash bound on chain	On-chain hash equals file SHA-256	T5
I4, I5	E	Malicious dependency runs at build time; ephemeral builder, filtered egress, no dataset	Payload runs in builder only—no secrets, no dataset; cache write denied	T2, T2b
I6, I7	T	Poisoned cache or modified artifact; mounted read-only at use	Build-time write denied (`Permission denied`); *residual:* wheels built from a malicious sdist are still cached	T2b
I7	I	Dataset exfiltration over the network; execution microVM has no interface	7/7 outbound channels fail, no `eth0`	T1, T2a
I7	I	Reads control-plane secrets or sibling data; no secrets, own disks only	Secrets absent from env; only own I/O disks visible	T1b, T4
CVM3 → CVM1	E	Executor pivots into the control plane; secrets and stores absent	Keys, IPFS, cache, executions absent (*residual:* shared net)	T4
CVM3	D	Residual state starves later runs; per-run caps (1 vCPU/512 MiB) and teardown	No socket, disk, or mount after 9+ runs	T3

Reading the evidence. Two points deserve emphasis beyond Table 1. First, the containment of the execution plane is *structural* rather than policy-based—the seven outbound probes fail because no network interface exists, not because a firewall rule denies them—and the plane that runs untrusted code is also the smallest first-party codebase (about 365 lines, against roughly 590 in the builder and 1,500 in the secret-holding control plane), so the boundary an attacker reaches first is the least code to audit. Second, the supply-chain test (T2) is the paper's thesis in miniature: a package seeded into the builder's cache is resolved, bundled, and runs its payload inside the execution microVM—yet finds no secrets and a blocked network. Malicious code inherits the containment of the plane that runs it.

Trivial, assumption-covered, and residual cells. *Trivial.* In-transit tampering and disclosure on I1 are handled by standard TLS; the ledger and store interactions (I2, I3) reduce to the trust assumptions of Section 4, and the rights and hashes they carry are public by construction; disclosure of the returned artifact (I6) is immaterial because the control plane commissioned it; and elevation of privilege through the marketplace API itself (I1) is an API-correctness assumption rather than a property the planes enforce. Two cells deserve a remark:

disclosure at the store (I3) is a mitigation we claim rather than assume—artifacts are encrypted before publication—and the design does not authenticate the package index (I5) beyond HTTPS, containing instead what its packages can do once resolved. *Acknowledged residuals and out of scope.* Four residuals are neither trivial nor fully closed, and we mark rather than claim them. First, the internal service channels (I4, I6–I8) are neither mutually authenticated nor segmented, so a compromised plane can impersonate an internal caller or tamper with the build bundle in transit; Section 6.4 measures what this actually grants an attacker, and internal mutual TLS would close it. Second, the read-only mount prevents a dependency's `setup.py` from writing to the wheel cache (T2b), but it does not make the cache *trustworthy*: a wheel legitimately built from a malicious sdist is persisted by design and may be reused by a later build of another buyer. What such a wheel reaches is still bounded by the plane that runs it—the builder, which holds no dataset and no secrets, and the networkless executor (T2, T2a)—so the residual is one of cache integrity rather than of dataset confidentiality; per-buyer partitioning and wheel provenance would close it. Third, a malicious dependency could exfiltrate the buyer's own application source over the filtered egress (I4/I5), but the provider's dataset never enters the build plane. Fourth, the result returned to the buyer (I8) is out of scope in two senses: this work bounds what a malicious application can *reach*, not what an authorized computation *reveals* about its inputs, nor whether a compromised executor returned a faithful result (Section 7). Denial of service beyond per-run state hygiene, and repudiation beyond the ledger-recorded actions of the I1/I2 row, are likewise out of scope.

Read against the requirements of Section 3, the per-interaction analysis exercises R5 (rows I1, I1/I2, and I3), R3 (rows I4/I5 and I6), R4 (both I7 rows and the D row), and R2 (the CVM3 $\rightarrow$ CVM1 row); R6 is not threat-facing and is exercised instead by Table 2. R1 is only partly measured, and we separate its two halves. Confidentiality against the *code the platform runs*—the adversary of our threat model—is what the I7 rows test: a malicious application finds no route out and no secret to take. Confidentiality against the *infrastructure operator* is not tested here; it rests on the confidential-computing substrate that the prototype substitutes away (Section 4), and artifact encryption protects assets at rest in the store but not the dataset as the control plane prepares it. That half of R1 is an assumption of the design, not a result of this evaluation.

6.3 Operational Cost

Table 2 consolidates the cost of the isolation strategy across architectures, drawing on two live campaigns. Execution latency does not separate A from B: the two differ by 67 ms in the 2026-06-11 campaign and by 11 ms on the benign comparator of the 2026-06-23 campaign, against a run-to-run standard deviation of 1.4 s measured over six repetitions of the same application. The A–B gap is thus two orders of magnitude below the noise of the measurement itself, so the data support the absence of a systematic cost from compartmentalization rather than an exact equality. The absolute value tracks host warmth—26.8 s

on the first campaign, 25.2 s on the warmer M0 host—and is dominated by the microVM life cycle (kernel boot, disk discovery, output-disk synchronization, guarded shutdown) rather than by the application, whose compute over sixty rows takes milliseconds.

Table 2. Comparative cost across the isolation spectrum: a no-isolation monolith (M0), a centralized deployment that adds ephemeral microVMs but colocates all assets (A), the compartmentalized design that also separates the planes (B), and the SGX-based original DNAT as a cited, qualitative reference.

Metric	M0	A	B	SGX (ref.)
Execution latency, benign app (s)[¶]	0.07	25.2	25.2	n/a
microVM boot/life-cycle overhead (s)[¶]	≈0	≈25.1	≈25.1	n/a
Execution latency, suite mean (s)[‡]	—	26.8	26.8	n/a
Build latency, no dependencies (s)[‡]	—	30.2	30.1	n/a
Build latency, `requests` +4 transitive (s)[‡]	—	34.9	38.2	n/a
Artifact size, no deps / `requests` (MiB)[‡]	—	16.0 / 19.6	16.0 / 19.6	n/a
On-chain gas, register / purchase (k)[§]	389/97	389/97	389/97	—
App reuse with deps, no recompile	yes	yes	yes	**no**

The microVM adds ≈25 s over M0's ≈0.07 s, since the application's compute over 60 rows is sub-second; that fixed cost is the price of a fresh, networkless microVM per run. A and B share the same Firecracker microVMs, so no cost difference is expected between them; the blast radius is where they diverge (Table 3). Latencies are single runs; the repeated-run spread of one application is 1.4 s (sd, n=6), which bounds how finely A and B can be told apart—the build gap in particular (n=1 per cell, and dependent on a public index) is not resolvable, and we draw no architectural conclusion from it. [¶]2026-06-23 same-host campaign. [‡]2026-06-11 A/B campaign (3 apps, warm host; M0 not included). [§]Gas is the EVM's unit of work (a plain transfer is 21k).

What that fixed overhead buys is visible in the same measurement: the no-isolation baseline (M0) exposes a working `eth0` and succeeds at network exfiltration, finds all six control-plane secrets in its environment, and persists its writes, whereas A and B block exfiltration, expose no secret, and leave no residual state.

The on-chain cost is likewise architecture-independent. Registering an asset costs ≈389k gas (a dataset with an empty bloom filter; an application ≈327k) and purchasing access ≈97k gas; a dataset's optional bloom filter adds ≈185k gas per 256 bytes. These figures are identical for M0, A, and B because the registry is the shared control plane—isolation lives in the build and execution planes, not in the contract. On a representative layer-2 deployment at 0.03 gwei with ETH at US$3,000, and setting aside the data-availability fee the rollup pays to its settlement layer, they amount to about US$0.035 to register an asset and under US$0.01 to purchase access—illustrative figures that float with the network, but that leave the marketplace inexpensive to operate.

6.4 A/B Comparison: Live Measurement of Both Environments

To remove any inference from the comparison, we ran the full suite against both deployments live and end to end: the compartmentalized system (Environment B) and the centralized baseline (Environment A, a single container colocating the same logic). Because both share the same Firecracker microVMs and the same access gate, the per-microVM guarantees are a constant—T1, T1b, T3 and T5 reproduce in both, and so does the operational cost—so the architectural variable surfaces only in the blast radius. That is where A and B diverge (Table 3, populated from live inspection rather than configuration): in B, the executor and the builder carry *none* of the control-plane secrets, and the sensitive stores (IPFS repository, wheel cache, executions store) are absent from their namespace, whereas in A those same secrets and stores sit in the single namespace that also runs the execution logic. The supply-chain test (T2) inherits the split: a compromised dependency's code executes, but in B inside a plane that holds neither secrets nor a route to them.

What the network residual still grants. Reachability is not exploitability, so we state what a compromised executor can *do* with the two ports it still reaches in B. The prototype does not authenticate its internal endpoints, so the control API can be driven as a confused deputy: an attacker on the executor can enumerate registered assets, read the stored output of previous executions, and ask the control plane to act with its wallet through the fixed set of operations the API exposes. This is a real residual, and it is why internal mutual TLS heads our future work. What it does *not* give is the platform: the encryption and wallet keys are never returned by any endpoint and remain in CVM1's environment, so the attacker can neither decrypt artifacts pulled from the store nor sign transactions of its own choosing, and the on-chain gate still refuses an execution for which no access was purchased (T5). In A the same attacker needs no network hop at all—both keys sit in its own environment and the stores on its filesystem—so it can decrypt every archived artifact and sign arbitrary transactions directly. The difference is therefore not the presence of a residual but its ceiling: a bounded, mediated set of API calls in B against unmediated custody of the platform's keys in A—*containment, not absolute security.*

6.5 Threats to Validity

The measurements were taken on a single host; absolute latencies will vary with hardware (a colder first campaign saw a 31.4 s execution mean and 103–131 s builds, with A and B again indistinguishable), and each cost figure is a single run, so we read them as orders of magnitude rather than precise separations. The containment and exfiltration results, by contrast, are structural—absence of a network interface, absence of secrets—and do not depend on timing. Our workload is also deliberately small: a 60-row dataset isolates the fixed microVM cost, but says nothing about how that cost amortizes over a long-running job, nor about where the per-run caps (1 vCPU, 512 MiB, fixed disk sizes) begin to bind. The network-reach residual bounds the blast-radius claim, with the consequences

Table 3. Reach of a compromised execution plane over control-plane assets: the compartmentalized design (B) versus the centralized baseline (A), from live inspection. "Isolated" = absent from the plane's namespace (the environment variable is not set, the store is not mounted); "Colocated" = same namespace as the execution logic.

Control-plane asset reachable from the execution plane	Env. B (proposed)	Env. A (baseline)
Asset-encryption key (env)	Isolated	Colocated
Wallet private key (env)	Isolated	Colocated
Service URLs: RPC, IPFS, builder, executor (env)	Isolated	Colocated
IPFS repository, wheel cache, executions store (filesystem)	Isolated	Colocated
IPFS API (port 5001, network)	Blocked	Reachable
Control API (3001) and blockchain RPC (8545), network	Reachable[†]	Reachable

[†]Residual risk in the single-host deployment: the three containers share one Docker network and the internal endpoints are unauthenticated, so a compromised executor can invoke the control API as a confused deputy (enumerate assets, read stored execution outputs) without ever holding its keys—see the discussion above. The defining assets of the blast radius—secrets and persistent stores—are nonetheless isolated in B and colocated in A.

and limits set out above, and confidentiality against the infrastructure operator (the second half of R1) is assumed rather than shown, for the substrate reason given next. Finally, the DoS analysis covers only per-run state hygiene, and the SGX column of Table 2 is qualitative rather than re-measured.

A more fundamental caveat concerns the execution substrate. The prototype runs each plane as a container with Firecracker microVMs rather than inside a hardware-backed confidential VM, so our measurements do not come from a genuinely confidential environment and we make no claim about confidential-computing overheads. This does not weaken the architectural argument: the properties we evaluate—privilege separation across planes, a networkless execution microVM, ephemeral teardown, and on-chain access control—belong to the architecture, not to the memory-encryption substrate. Running the same planes inside confidential VMs (e.g., AMD SEV-SNP [1]) would add attestation and memory-encryption costs—so the cost figures above would not carry over unchanged—but would preserve the containment and blast-radius results, which is why we treat confidential execution as a substrate substitution rather than an architectural change.

7 Related Work

A line of work uses trusted execution and distributed ledgers to control how sensitive data is used after it is handed to an untrusted consumer. The original DNAT [6] pioneered this combination for *auditable traceability*, recording data-and-application usage in a tamper-evident ledger while executing the application inside an SGX enclave; it assumes, however, that the application has

been pre-verified as non-malicious by the data owner or a trusted third party. PrivacyGuard [9] similarly enforces usage contracts on chain and runs the computation in an attested off-chain enclave, again taking the processing code as ratified in advance. The closest work, VDDPI [8], removes that assumption: it verifies the consumer's code *mechanically*—through a mandatory template, an allow-listed function set, and static taint analysis executed by smart contracts—so that a provider can learn, before releasing data, whether the code discloses its inputs, and it binds the analysed code to the running enclave through the SGX measurement.

Our work is complementary to that verification axis rather than competing with it. VDDPI answers *what* an authorized computation may reveal about its inputs—the concern we place out of scope (Section 6)—but pays for it with a restricted programming model: applications must fit the template and the allow-list, so arbitrary dependencies and unmodified code are not supported. We instead target *how* partially trusted code is run, treating dependency resolution (which VDDPI forbids outright) as a first-class supply-chain threat. The two are composable: a code-verification stage in the spirit of VDDPI could be attached to our control plane without altering the plane boundaries.

8 Conclusion

This paper framed the proposed system as a layered containment architecture for a confidential data marketplace and tested that framing against the running prototype. The central design move—separating control, build, and execution into planes with different network and persistence properties—was evaluated not as an abstract claim of trustlessness but as a measurable reduction in blast radius relative to a centralized baseline differing only in the colocation of assets.

The experimental results support that argument. Execution-time code cannot open outbound channels: the probed channels failed in the network stack itself—unimplemented socket calls and failed name resolution—rather than at a policy that could be switched off. Ephemeral state does not survive, and the assets that define the blast radius (the encryption and wallet keys, the IPFS repository, the wheel cache, and the executions store) are absent from the compromised executor in the compartmentalized design, whereas they are colocated in the baseline. Execution is gated on chain, and the isolation costs roughly twenty-five seconds per run—a cost compartmentalization itself does not increase.

The limitations are equally concrete: the planes share one network and do not authenticate each other, so a compromised executor can still drive the control API as a confused deputy—though never obtaining the keys to decrypt artifacts or sign transactions; the wheel cache is not yet a trustworthy artifact store; dataset confidentiality depends on preparation outside the encrypted-artifact path; and no attestation flow yet verifies each component's software identity. The design is nonetheless a credible high-assurance baseline, and future work—internal mutual TLS, wheel and dataset provenance, and attestation—will let the containment shown here be verified remotely rather than assumed.

Declaration on the Use of Generative AI

During the preparation of this work, the authors used generative AI tools (large language models) to assist with drafting and editing parts of the manuscript and with the development and debugging of the prototype implementation and evaluation code. The authors reviewed, verified, and edited all resulting content and take full responsibility for the entire content of this paper. No AI tool is listed as an author, and no AI tool was used to generate or fabricate experimental data, results, or scientific claims.

References

1. Advanced Micro Devices, Inc.: AMD SEV-SNP: Strengthening VM isolation with integrity protection and more. Tech. rep., Advanced Micro Devices, Inc. (2020)
2. Agache, A., Brooker, M., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., Popa, D.M.: Firecracker: Lightweight virtualization for serverless applications. In: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20). pp. 419–434. USENIX Association (2020)
3. Benet, J.: IPFS - content addressed, versioned, P2P file system. CoRR **abs/1407.3561** (2014), https://arxiv.org/abs/1407.3561
4. Buterin, V.: A next-generation smart contract and decentralized application platform. Ethereum White Paper (2014), https://ethereum.org/en/whitepaper/
5. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. White Paper (2008), https://bitcoin.org/bitcoin.pdf
6. Nascimento Jr., J.R., Nunes, J.B.S., Falcão, E.L., Sampaio, L., Brito, A.: On the tracking of sensitive data and confidential executions. In: Proceedings of the 14th ACM International Conference on Distributed and Event-based Systems (DEBS '20). pp. 1–10. ACM, Virtual Event, QC, Canada (2020). https://doi.org/10.1145/3401025.3404097
7. Shostack, A.: Threat Modeling: Designing for Security. John Wiley & Sons, Indianapolis, IN (2014)
8. Tokuda, S., Kakei, S., Shiraishi, Y., Saito, S.: VDDPI: Verifiable decentralized data processing infrastructure for data usage control and confidential data processing. IEEE Transactions on Dependable and Secure Computing **23**(3), 6068–6084 (2026). https://doi.org/10.1109/TDSC.2026.3658309
9. Xiao, Y., Zhang, N., Li, J., Lou, W., Hou, Y.T.: PrivacyGuard: Enforcing private data usage control with blockchain and attested off-chain contract execution. In: Computer Security – ESORICS 2020. pp. 610–629. Springer (2020)